

University of Wolverhampton
School of Mathematics and Computer Science

Student Number:

Name:

6CS005 High Performance Computing Week 10 Workshop

Revision on CUDA

Tasks – CUDA Block, Grid and Thread

From what you have learnt in the Lecture, using only multiple blocks and threads, produce the following programs:

Revision on CUDA

Tasks – CUDA Block, Grid and Thread

From what you have learnt in the Lecture, using only multiple blocks and threads, produce the following programs:

1. The below cuda program attempts to launch a GPU kernel that calls a function to print "Hello World".

```
#include <stdio.h>

void displayHelloWorld(){
    printf("Hello World\n");
}

__global__ void display(){
    displayHelloWorld();
}

int main(){
    display<<<1,1>>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

- a. Does the code work ?

b. Modify the code such that the function `displayHelloWorld` is callable inside the kernel.

2. Run and observe the output of the following code:

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void display() {
    int threadID = threadIdx.y;
    int blockID = blockIdx.x;
    printf("Block %d -> ThreadId = %d\n", blockID, threadID);
}

int main() {
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(5, 1, 1);
    display<<<gridSize, blockSize>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

Expected Output:

```
Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 1
Block 0 -> ThreadId = 2
Block 0 -> ThreadId = 3
Block 0 -> ThreadId = 4
```

- a. Does the actual output match the expected output? Identify the problem.
 - b. Modify the code so that it produces the correct output and explain your changes.
3. In the code above, the block dimension is specified as **dim3 blockSize(5, 1, 1)**. Which means that the block has:
- 5 threads in the x-dimension
 - 1 thread in the y-dimension
 - 1 thread in the z-dimension

This typically means that the block we created is a **1D block** with 5 threads in the x-direction.

- a. Modify the code so that it creates a 2D block instead of a 1D block. Also print thread ids along its corresponding directions/axis. For example if you are using a 2D block specified as **dim3 blockSize(5, 1, 3)** print thread ids along **x** and **z** direction/axis similar to the one given below:

```
Block : 0 -> ThreadX = 0, ThreadZ = 0
Block : 0 -> ThreadX = 1, ThreadZ = 0
Block : 0 -> ThreadX = 2, ThreadZ = 0
Block : 0 -> ThreadX = 3, ThreadZ = 0
Block : 0 -> ThreadX = 4, ThreadZ = 0
Block : 0 -> ThreadX = 0, ThreadZ = 1
Block : 0 -> ThreadX = 1, ThreadZ = 1
Block : 0 -> ThreadX = 2, ThreadZ = 1
Block : 0 -> ThreadX = 3, ThreadZ = 1
Block : 0 -> ThreadX = 4, ThreadZ = 1
Block : 0 -> ThreadX = 0, ThreadZ = 2
Block : 0 -> ThreadX = 1, ThreadZ = 2
Block : 0 -> ThreadX = 2, ThreadZ = 2
Block : 0 -> ThreadX = 3, ThreadZ = 2
Block : 0 -> ThreadX = 4, ThreadZ = 2
```

- b. Modify the code again so that it creates a 3D block instead of 2D block. Also print thread ids along its corresponding directions/axis.
4. The below code takes an integer **n** from the user and prints all numbers between **0 to n - 1** using the kernel function.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void displayIdx(){
    int tid = threadIdx.x;
    printf("%d\n", tid);
}

int main(){
    int n;
    printf("Enter n : ");
    scanf("%d", &n);
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(n, 1, 1);
```

```

displayIdx<<<gridSize, blockSize>>>();
cudaDeviceSynchronize();
return 0;
}

```

- a. Does the code work when the user enters n as 1025 ? Explain.
- b. Does the code still work when changing the dim3 **blockSize(n, 1, 1)** line of code to **dim3 blockSize(n, n, 1)** ? Modify the code so that the values are correctly printed out.

Hint:

```

__global__ void displayIdx(){
    int tid = threadIdx.x; // Try changing this line of code.
    printf("%d\n", tid);
}

```

Note:

If n = 5, the code should print the value 0 to 24.

- c.
 - d. Now change the **dim3 blockSize(n, n, 1)** to **dim3 blockSize(n, n, n)** and also modify the code so that it correctly prints out the value. Example if n = 5, the code should print the value 0 to 124.
6. Create a CUDA program to solve quadratics (find the roots). You will be given a text file containing 3 integers on each row which represent a, b and c for the following formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

